

TEAM A — AirMonitor

Dominant Smell: *Organizational Silo Effect*

Secondary: *Radio Silence Effect (unique boundary spanner)*

Project Context

AirMonitor is an open-source framework for orchestrating environmental data pipelines. As the codebase evolved, the community split into two loosely coordinated groups: the *pipeline-core team* and the *data-visualization team*. Despite working on interdependent modules, the two groups rarely communicate directly. Almost all cross-team communication flows exclusively through **Dev_B**, an experienced but overloaded maintainer. This has created a fragile organizational structure where the absence or overload of a single developer halts all cross-group knowledge transfer.

Communication Snippets

None

[Slack #pipeline-core]

Dev_A: API for step orchestration changed yesterday.

Dev_B: I'll tell the UI team.

Dev_C: Shouldn't they be in this channel too?

Dev_B: No, I'll handle it. ← # RadioSilence (unique boundary spanner)

None

[Slack #ui-dev]

Dev_H: Charts broke again due to backend schema updates.

Dev_G: Did anyone notify us?

Dev_H: Only heard from Dev_B last time... still waiting. ← # RadioSilence

None

[GitHub Issue #3481 – “UI not syncing with pipeline changes”]
Dev_G: We need clarification on new scheduler fields.
Dev_B: I’ll collect answers from core and report back. ← #
RadioSilence
(no direct replies from core developers)

None

[Internal Note – Weekly Integration Meeting]
Dev_B: I updated the UI team about API removals.
Dev_A: Why not invite them to our meeting?
Dev_B: It’s easier if I relay messages. ← # OrganizationalSilo
+ RadioSilence

None

[PR #4410 – “Connector alignment for v3.1”]
Dev_H: Need confirmation about new pipeline-core flags.
Dev_B: I’ll ask the others and return with answers. ← #
RadioSilence
(no messages from Dev_A or Dev_C in the thread)

None

[Slack DM – Dev_H → Dev_B]
Dev_H: Sorry to DM, but we only get updates through you.
Dev_B: Yeah, it’s messy. I’ll try my best. ← #
BoundarySpannerOverload

Developer Network

None

(Core Team)
Dev_A -- Dev_C
 \ /
 \ /
 \ /

```
      Dev_B ← unique bridge (boundary spanner)
      |
Dev_F --- Dev_G --- Dev_H
      (UI Team)
```

- **Two dense internal clusters** (core vs UI)
- **Only ONE cross-team edge:** Dev_B ↔ UI developers

This matches the precise definition of **Radio Silence**: *a single individual interposes themselves into every formal interaction between sub-communities.*

Metrics

Network Density = 0.23

Interpretation:

Only 23% of possible communication links exist → sparse network typical of siloed structures.

Modularity = 0.74 (Louvain method)

Concept: Measures how strongly a network divides into sub-communities.

Interpretation:

High modularity (>.7) = strong, rigid silos.

3. Betweenness Centrality(Dev_B) = 0.68

Interpretation:

Dev_B appears in **nearly all shortest paths** between core and UI developers → *unique boundary spanner*.

4. Edge Redundancy = 0

Interpretation:

There are **zero** backup communication links → Radio Silence confirmed.

Observations

- The **only cross-team connector is Dev_B**, who acts as the forced mediator for all interactions.
 - If Dev_B is busy or unavailable, **communication between teams *completely stops***.
 - UI developers repeatedly report receiving information only from Dev_B.
 - Attempts to add others into cross-team communication are shut down or ignored.
 - The system is **structurally fragile** and creates **accumulating social debt**.
-

Refactoring Plan

Recommended Refactorings

- Introduce **multiple cross-team liaisons** (rotate monthly).
- Establish a **shared cross-team Slack channel** instead of two isolated ones.
- Hold **joint planning meetings** between core and UI (no single mediator).
- Create a **shared integration board** visible to both teams.
- Implement **documentation agreements** for breaking changes.

TEAM B — TensorML

Dominant Smell: *Black Cloud Effect*

Secondary: *Radio Silence Effect* (in the *structural* sense: a single bottleneck handling communication in overloaded channels)

Project Context

TensorML is a widely used machine learning library maintained by a fast-moving, globally distributed team. Over time, communication became fragmented across Slack, GitHub issues, Discord, and mailing lists, with no consistent structure or process for discussion. As contributors increased, discussions became extremely verbose, repetitive, and emotionally charged — creating an atmosphere of **continuous information overload**. A small set of senior developers, particularly **Dev_A**, dominate these channels and act as gatekeepers for all design decisions. This high-volume, unstructured communication environment has created sustained cognitive pressure and social fatigue in the developer community.

Communication Snippets

None

[Slack #model-design – 126 messages in 2 hours]

Dev_A: Let's finalize optimizer API today.

Dev_K: Posting my proposal again; it got buried last week. ← #BlackCloud (overload)

Dev_A: Please scroll up.

Dev_K: There are hundreds of messages! ← # BlackCloud

None

[GitHub Issue #2210 – “Confusion around loss function changes”]

Dev_M: I'm lost in discussions. Where's the final decision?

Dev_A: It's explained in Slack thread.

Dev_M: Which thread? There are 14. ← # BlackCloud

None

[Discord #random-dev-talk]

Dev_P: Anyone know if GELU is approved?

Dev_J: Dev_A said no somewhere yesterday...

Dev_L: I think I saw a yes two days ago? ← # BlackCloud
(contradictory async chatter)

None

[PR #587 - "Add GELU activation"]

Dev_H: Implemented GELU per user requests.

Dev_A: Not needed. Closing. ← # BlackCloud (dismissive +
chaotic decision making)

Dev_H: Can we discuss?

(no further replies) ← # RadioSilence (structural: only Dev_A
decides)

None

[Slack #core-dev]

Dev_L: Should we split discussion channels?

Dev_A: Too complicated.

Dev_L: But people are leaving because it's overwhelming. ← #BlackCloud (burnout)

Developer Network

None

```
Dev_H      Dev_J
  \        /
   \      /
    \    /
    Dev_A -- Dev_L -- Dev_P -- Dev_K
      \                /
       -- Dev_M ----
```

- **High density inside the core group** (everyone talks to everyone constantly)
- **Extremely high message volume**
- **Dev_A** centrally controls decision flow → structural bottleneck

This is the exact “storm cloud” pattern described in the *Black Cloud Effect*: *excessive, unstructured communication that overwhelms cognitive capacity and emotional bandwidth*.

Metrics

1. Message Volume = ~250 messages/day

How computed: Count unique messages across Slack, GitHub, Discord in a 24h window.

Interpretation: Extremely high load → strong Black Cloud indicator.

2. Density = 0.41

Interpretation: High density amplifies information overload.

3. Sentiment Variance = ± 0.76

Interpretation: Emotional instability in communication → typical of Black Cloud.

4. Betweenness(Dev_A) = 0.55

Interpretation: Dev_A acts as a decision filter → contributes to Radio Silence structurally.

Observations

- High communication volume across multiple channels.
 - Decisions lost in unstructured threads → cognitive overload.
 - One senior dev (Dev_A) dominates decisions → structural bottleneck.
 - Contributors express confusion, frustration, and burnout.
-

Refactoring Plan

Recommended Refactorings

- Introduce **topic-based channels** with clear ownership.
- Hold **weekly synchronous decision meetings** (recorded + summarized).
- Use **issue templates** and **decision logs**.
- Rotate **discussion facilitators** to distribute cognitive load.

TEAM C — DataWeaver

Dominant Smell: *Organizational Silo Effect*

Secondary: *Radio Silence Effect* (unique bridge roles)

Project Context

DataWeaver is a visual analytics framework consisting of two main sub-teams: frontend visualization and backend API. The two groups evolved independently over years, developing their own cultures, tools, and meeting routines. Coordination is done almost exclusively by **Dev_Q**, who drifts between both teams. This has caused repeated integration failures, duplicated effort, and inconsistent design decisions across modules. Attempts to unify communication structures have repeatedly stalled due to conflicting priorities.

Communication Snippets

None

[Slack #frontend]

Dev_L: API changed again yesterday?

Dev_P: Yes, backend discussed it.

Dev_L: Why weren't we included? ← # Silo

Dev_Q: I can update you later. ← # RadioSilence (unique liaison)

None

[GitHub Issue #902 – “Chart rendering fails after update”]

Dev_M: Backend update broke tooltips.

Dev_P: That's frontend's responsibility.

Dev_M: We didn't know the schema changed! ← # SiloBlame

None

[Slack #backend]

Dev_P: UI keeps complaining about schema changes.

Dev_R: They should read our commit history.

Dev_Q: They can't if you don't document the changes. ← # Silo Conflict

None

[PR #1180 – “Add unified schema file”]

Dev_Q: Proposed shared API schema to prevent drift.

(no replies for a week) ← # RadioSilence (ignored boundary spanner)

None

[Slack DM – Dev_Q → Dev_L]

Dev_Q: Backend changed validation rules again.

Dev_L: Why only you tell us? Can't they talk directly? ← # RadioSilence

None

[Standup Transcript]

Dev_R: UI should follow backend pace.

Dev_L: Hard to follow when we hear changes second-hand.

Moderator: Let's include both teams next time.

(never happens) ← # SiloPersistence

Developer Network

None

Frontend: Dev_L -- Dev_M -- Dev_Q

\

Backend: Dev_P -- Dev_R -- Dev_T

Single cross-team connection: Dev_Q ↔ Dev_P/Dev_R.

Metrics

1. Modularity = 0.81

High Q indicates **very strong siloing**

2. Betweenness(Dev_Q) = 0.67

Dev_Q appears in almost all cross-team paths → classic Radio Silence.

3. Density = 0.18

Sparse network → minimal collaboration.

4. Edge Redundancy = 0

No parallel cross-team links.

Observations

- Two heavily isolated sub-teams
- Dev_Q forced to mediate everything
- Breakdowns in coordination
- Silo blames in communication

Refactoring Plan

Recommended Refactorings

- Create **joint architecture sync meetings**
- Add **additional cross-team liaisons**
- Build **shared API documentation**
- Establish **integration testing pipeline**

TEAM D — AutoServe

Dominant Smell: *Lone Wolf Effect*

Project Context

AutoServe provides automated deployment infrastructure for microservices. One long-term contributor, **Dev_E**, has gradually assumed sole ownership of the deployment module. Over time they began rejecting reviews, working in isolation, and merging changes independently. Other maintainers have grown hesitant to modify the module due to its complexity and Dev_E's protective behavior. This concentration of knowledge creates significant long-term social and technical risk.

Communication Snippets

None

[PR #145 – “Refactor deploy script”]

Dev_E: Cleaned up logic; merging soon.

Dev_B: Want feedback?

Dev_E: No, doing it myself. ← # LoneWolf

None

[Issue #901 – “Monitoring restart failure”]

Dev_C: Any update?

Dev_E: Already fixed locally.

Dev_C: Can you push it?

Dev_E: When I get to it. ← # LoneWolf (gatekeeping)

None

[Slack #devops]

Dev_D: Who understands deploy module?

Dev_B: Only Dev_E.

Dev_D: That's... not great. ← # LoneWolfRisk

None

[GitHub Activity Log]

14 self-merged PRs by Dev_E in last 30 days ← # LoneWolfEvidence

None

[Slack DM]

Dev_B → Dev_E: Can we help with deploy script?

Dev_E: It's faster if I stay in control. ← # LoneWolf

None

[Standup Notes]

Team lead: Deployment knowledge needs spreading.

Dev_E: I prefer to manage it. ← # LoneWolf

Developer Network

None

Dev_B -- Dev_C -- Dev_D

\

\

(Dev_E) ← isolated, minimal edges

Metrics

1. Degree(Dev_E) = 1

Only one collaborator → isolation.

2. Code Ownership = 92%

Extremely high → single point of failure.

3. Review Participation = 0%

No received or given reviews.

4. Clustering Coefficient = 0.0

No triads → structural isolation.

Observations

- Near-complete module monopoly
 - Refusal to collaborate
 - High-risk social debt scenario
-

Refactoring Plan

Recommended Refactorings

- Introduce **pair programming rotation**
- Require **minimum two-reviewer merges**
- Create **onboarding pathway** for deployment module
- Run **knowledge-sharing workshops**

TEAM E — SecureAI

Dominant Smell: *Radio Silence Effect*

Secondary: *Black Cloud Effect*

Project Context

SecureAI builds critical security tooling for machine learning pipelines. Its core maintainers are overloaded and responsible for reviewing highly sensitive crypto-related contributions. Most external and even internal communication passes through **Dev_S**, who is the de facto gatekeeper for all reviews and design approvals. Meanwhile, the project's Slack channels are filled with dozens of asynchronous threads, security warnings, bot notifications, and long debates about encryption standards. The result is a fragile structure: **a single communication bottleneck inside an overwhelming Black Cloud environment.**

Communication Snippets

None

[PR #78 – “Refactor encryption module”]

Dev_K: Requesting review.

Dev_S: I will check soon.

(others stay silent) ← # RadioSilence (Dev_S is only reviewer)

None

[Slack #security-team – 180 msgs/day]

Dev_V: Can someone triage open PRs?

Dev_S: Not today, too much noise. ← # BlackCloud

Dev_K: Then who can?

Dev_S: Only me. ← # RadioSilence

None

[GitHub Issue #3101 – “Key rotation logic inconsistent”]

Dev_U: We need consensus.

(no replies except Dev_S two days later) ← # RadioSilence

None

[Slack #alerts]

Security bot: *15 new CVE alerts*

Security bot: *Build rate-limited*

Dev_T: Too many alerts to follow. ← # BlackCloud

None

[Slack DM – Dev_K → Dev_V]

Dev_K: Why does everything wait for Dev_S?

Dev_V: That’s how it’s always been... ← # RadioSilence
(structural)

None

[Standup Notes]

Dev_S: I'm the only one who can approve crypto changes.

Team Lead: We need backups.

(no volunteers) ← # RadioSilence Reinforced

Developer Network

None

```
Dev_R -- Dev_S -- Dev_U -- Dev_V
      |
      Dev_K
```

Notes:

- Dev_S is the *only* bridge
- Others cluster around them without cross-links

Metrics

1. Betweenness(Dev_S) = 0.71

High → classic single boundary spanner.

2. Edge Redundancy = 0

No alternative reviewers or connectors.

3. Density = 0.25

Sparse network with central bottleneck.

4. Message Volume = 150+/day

Count across Slack channels → Black Cloud.

5. Modularity = 0.66

Clear separation between contributors and maintainers.

Observations

- Dev_S mediates *all* communication.
 - Info overload contributes to burnout.
 - No backup reviewers for security-critical work.
-

Refactoring Plan

Recommended Refactorings

- Add **secondary crypto reviewers**
- Introduce **weekly triage meetings**
- Reduce redundant bot notifications
- Establish **shared threat model documentation**

Metrics and Definitions

- **Network Density**
How interconnected the network is. Low density means team members rarely communicate.
- **Modularity**
How strongly the network is divided into sub-groups. High modularity that the network is strongly fragmented
- **Betweenness Centrality**
How often a person acts as a bridge between others. High values suggest a central or influential role.
- **Edge Redundancy**
Whether there are multiple paths for communication. Zero redundancy means a single point of failure.
- **Message Volume**
The number of messages exchanged over time. High volume may reflect overload or chaos.
- **Sentiment Variance**
How much the emotional tone fluctuates in communication. High variance indicates instability.
- **Degree (or Degree Centrality)**
How many direct connections a person has. Low degree means isolation.
- **Code Ownership**
The extent to which one person controls the code.
- **Review Participation**
How often someone takes part in code reviews. Zero participation implies a lack of collaboration.
- **Clustering Coefficient**
How well a person's neighbors are connected to each other. Low clustering means collaborators aren't talking to one another.